PM2 Clean-Code Handbuch

Über dieses Dokument

Dieses Dokument listet die in PROG2/PM2 einzuhaltenden Clean-Code Regeln. Es ist eine überarbeitete Fassung des in PM1/PROG1 verwendeten Materials (entsprechend markiert). Diese Clean-Code Regeln sind für das ganze Studium an der ZHAW gültig und werden, sofern nicht anders vermerkt, konsequent eingefordert. Die vorhandenen Beispiele sollen die Clean-Code-Regeln illustrieren. Sie dienen nicht dazu, neuen Stoff über den in der Vorlesung behandelten einzuführen.

Neue Elemente im Vergleich zu PM1 wurden mit ✚ markiert (über 20 Einträge).

Bestimmte Elemente werden nur in bestimmten Projekten (bezeichnet mit L1, L2, L3 und L4) bewertet.

Inhaltsverzeichnis

Über dieses Dokument . . . 1
Review-Kriterien im Detail . . . 3
- Lesbarkeit . . . 3
  - Java-Namenskonventionen werden eingehalten (L1) . . . 3
  - Code ist aufgeräumt (L1) . . . 3
  - Sinnvolle Namensgebung (L1) . . . 7
  - Methodengestaltung (L1,L3) . . . 8
- Klassengestaltung . . . 10
  - Klassenoberfläche ist minimal (L1) . . . 10
  - Sinnvolle Abstraktionsebene (L1) . . . 10
  - Klassen stehen für sich allein (L2) . . . 10
- Testing . . . 12
  - Ein sinnvoller Unit-Test pro Klasse ist vorhanden (L1) . . . 12
  - Logik mit Tests abgedeckt (L1) . . . 12
  - Kollaborateure als Konstruktor-Argumente (L2) . . . 12
  - Parametrisieren von Tests (L3) ✚ . . . 14
  - Mock-Objekte statt externe Kollaborateure (L3) . . . 14
  - Mockito (L4) ✚ . . . 16
- Wartbarkeit . . . 17
  - Vernünftiger Einsatz von Feldern (L1) . . . 17
  - Interfaces statt konkrete Typen (L1) . . . 17
  - Angemessene Verwendung von Collections (L1) . . . 17
  - Klassen, ENUMs statt primitive Typen oder Strings (L2) . . . 18
  - Ausnahmen werden behandelt (L2) ✚ . . . 19
  - Logging anstelle von System.out (L3) ✚ . . . 20
  - Anwender erhält sinnvolle Fehlermeldung (L3) ✚ . . . 22
  - Javadoc (L1) . . . 22
  - Dokumentieren mit Beispielen (L2) ✚ . . . 23
- Korrektheit . . . 25

Object-Klassenvertrag wird eingehalten (L1) . . . 25
Eingabeüberprüfung (L1) ✚ . . . 25
Null-(Leer-)Objekt anstelle von null als Rückgabewert (L2) . . . 26
Optional-Container anstelle von null Rückgabewert (L2) ✚ . . . 27
Ressourcen werden geschlossen (L3) ✚ . . . 28
Streams sind seiteneffektfrei (L3) ✚ . . . 30
Vermeiden Sie Lecks durch Referenzen (L3) ✚ . . . 30
Varia . . . 32
Frühe, häufige Commits (L1) ✚ . . . 32
Gute Commits (L2) ✚ . . . 32
Verwenden von Feature-Branches und Pull-Requests (L2) ✚ . . . 33
Standard-Java-Verzeichnisstruktur (L2) ✚ . . . 33
Vernünftige Objektlebenszeiten (L2) ✚ . . . 33
Funktional über Prozedural . . . 35
Favor Format over concatenation (L2) ✚ . . . 35
Favor For each over For loops (L2) ✚ . . . 35
Favor functional over imperative style (L4) ✚ . . . 36
Use collect for terminating complex streams (L4) ✚ . . . 36
Literatur . . . 38

Review-Kriterien im Detail

Lesbarkeit

Java-Namenskonventionen werden eingehalten (L1)

- Klassennamen sind Upper Camel Case (`MySpecialClass`)
- Methodennamen, Variablennamen und Feldnamen sind Lower Camel Case (`someMethod()`)
- Konstanten sind Grossbuchstaben (`ABSOLUTE_NULL`)

Java-Klasse mit eingehaltenen Namenskonventionen ☺

```
public class Temperature {

    private static final BigDecimal KELVIN_CELSIUS_DIFF = BigDecimal.valueOf(273.15);

    private final BigDecimal value;

    public Temperature(BigDecimal degreesKelvin) {
        this.value = degreesKelvin;
    }

    public BigDecimal toCelsius() {
        return value.add(KELVIN_CELSIUS_DIFF);
    }
}
```

Code ist aufgeräumt (L1)

Kein toter Code ist vorhanden

Entfernen Sie auskommentierten Code umgehend. Er reduziert die Lesbarkeit des Codes und wird spätestens nach der nächsten Umbenennung einer Variablen nutzlos. Falls Sie fürchten, die alte Implementierung noch einmal zu brauchen: Dafür ist die Versionskontrolle da.

Klasse mit totem Code ☹

```
public class Temperature {
    public static BigDecimal toCelsius(BigDecimal degreesKelvin) {
        // BigDecimal.valueOf(value.floatValue() + 273.15) ①
        return degreesKelvin.add(BigDecimal.valueOf(273.15));
    }
}
```

① Toter Code. Ist `value` dasselbe wie `degreesKelvin`?

Sinnvolle Kommentare

Redundanter Kommentar ☹

```
public class Temperature {
    public static BigDecimal toCelsius(BigDecimal degreesKelvin) {
        // Add 273.15 to the value in Kelvin to get the temperature in celsius! ①
        return degreesKelvin.add(BigDecimal.valueOf(273.15));
    }
}
```

① Erzählt nur, was der Code in der nächsten Zeile tut. Aber warum tut er, was er tut?

Anstatt zu beschreiben, was der Code tut, fügen Sie nützliche Informationen hinzu, die Ihrem zukünfti-

gen Ich beim Verständnis der Passage helfen:

- Berechnungsformeln
- Links zu Online-Referenzen
- Begründungen

Weiterführender Kommentar ☺

```
public class Temperature {
    public static BigDecimal toCelsius(BigDecimal degreesKelvin) {
        // https://en.wikipedia.org/wiki/Conversion_of_units_of_temperature
        return degreesKelvin.add(BigDecimal.valueOf(273.15));
    }
}
```

Erklärender Kommentar aus OkHttp ☺

```
public URI uri() {
    String uri = newBuilder().reencodeForUri().toString();
    try {
      return new URI(uri);
    } catch (URISyntaxException e) {
      // Unlikely edge case: the URI has a forbidden character in the fragment. Strip it
& retry.
      try {
        String stripped = uri.replaceAll("[\\u0000-\\u001F\\u007F-
\\u009F\\p{javaWhitespace}]", "");
        return URI.create(stripped);
      } catch (Exception e1) {
        throw new RuntimeException(e); // Unexpected! ①
      }
    }
  }
```

① Schreiben Sie nicht bei jeder `RuntimeException` oder einer Ableitung davon den Kommentar "Unexpected!" hin. Dies ist in diesem Beispiel nur aufgrund des Kontextes angebracht.

Importieren Sie nur, was Sie brauchen

Importieren Sie nur die Namen, die Sie in der aktuellen Datei tatsächlich brauchen. Unnötige Imports - dazu gehören auch die sogenannten Star Imports, wie `import java.util.*;` - sorgen zwar nur für einen kleinen Performance-Verlust und das auch nur beim Kompilieren, können Ihnen aber Wartungsprobleme bereiten. Folgender Code lief in Java 1.1 problemlos. Beim Upgrade auf Java 1.2 versagt er, auch wenn der Code nicht angefasst wurde. Warum?

Drohendes Unglück ☹

```
import java.math.BigDecimal;
import java.util.*;
import java.awt.*;

public class Test {

    public static void main(String[] args) {
        List list = new List();
    }
}
```

Der Typ `List` ist `java.awt.List`, in Java 1.1 der einzig verfügbare Listen-Typ. Mit Java 1.2 kommt mit `java.util.List` plötzlich ein zweiter Typ `List` dazu, womit der Code mehrdeutig wird.

Verzichten Sie auf Copy & Paste

Bevor Sie Code kopieren und an einer anderen Stelle einfügen, überlegen Sie sich, ob Sie nicht eine Methode schreiben und diese referenzieren können.

Besonders geeignete Kandidaten:

- Berechnungen bzw. Zusammensetzung von Zeichenketten (z.B. Vor- und Nachname)
- Objekte erstellen aus SQL-`ResultSet` und umgekehrt
- Hilfsmethoden, z.B. zum Ausgeben eines Datums

Vermeiden Sie unnötige Vergleiche/Negationen ➕

Booleans sollten in Bedingungen nicht verglichen werden ... ☹

```
class Laboratory {
    Microscope microscope;
    Result analyze(Sample sample) {
        if (microscope.isInorganic(sample) == true) {
            return Result.INORGANIC;
        } else {
            return analyzeOrganic(sample);
        }
    }
    private Result analyzeOrganic(Sample sample) {
        if (microscope.isHumanoid(sample) == false) {
            return Result.ALIEN;
        } else {
            return Result.HUMANOID;
        }
    }
    private Result analyzeOrganicWithNegation(Sample sample) {
        if (microscope.isHumanoid(sample) != false) {
            return Result.HUMANOID;
        } else {
            return Result.ALIEN;
        }
    }
}
```

Vergleiche mit Boolean in der If-Bedingung sind redundant:

```
class Laboratory {
    Microscope microscope;
    Result analyze(Sample sample) {
        if (microscope.isInorganic(sample)) { ①
            return Result.INORGANIC;
        } else {
            return analyzeOrganic(sample);
        }
    }
    private Result analyzeOrganicWithNegation(Sample sample) {
        if (microscope.isHumanoid(sample)) { ②
            return Result.HUMANOID;
        } else {
            return Result.ALIEN;
        }
    }
```

```
}
```

① Das `== true` können Sie einfach weglassen.
② Stellen Sie die Reihenfolge des then- und des else-Zweiges um, um eine Negation zu verhindern.

Ziehen Sie Konstanten Magic Numbers vor

Magic Numbers sind (meist) Zahlen, die plötzlich im Code auftauchen — und niemand weiss, was sie bedeuten und warum gerade der angegebene Wert der Richtige ist.

Temperaturkonversion mit Magic Number ☹

```
public class Temperature {
    public static BigDecimal toCelsius(BigDecimal degreesKelvin) {
        return degreesKelvin.add(BigDecimal.valueOf(273.15)); ①
    }
}
```

① 273.15 ist eine Magic Number

Erklären Sie die Magic Numbers mit einem Kommentar oder — meist besser — durch eine Konstante, da Sie dann immer denselben Wert referenzieren und ihn so nur einmal zu ändern brauchen.

Magic Number als Konstante ausgedrückt ☺

```
public class Temperature {

    private static final BigDecimal KELVIN_CELSIUS_DIFF = BigDecimal.valueOf(273.15); ①

    public static BigDecimal toCelsius(BigDecimal degreesKelvin) {
        return degreesKelvin.add(KELVIN_CELSIUS_DIFF);
    }
}
```

① Benannte Konstante

Formatieren Sie Code konsistent

Ob Sie Tabs oder Leerschläge zum Einrücken des Codes verwenden, spielt keine grosse Rolle. Ebenso wenig, ob Sie Blöcke auf derselben Zeile mit `if`, `for` usw. beginnen oder in der Zeile darunter. Nur tun Sie es konsistent. Dies verbessert die Lesbarkeit und sorgt für minimalen "Lärm" durch Formatierungsänderungen beim Code Review. Erstellen Sie jeweils eigene commits für Formatierungänderung und für Logikänderung, bitte nicht mischen.

Verwenden Sie die IDE, um den Code automatisch formatieren zu lassen.

IntelliJ

Code › Reformat Code → Windows `Ctrl` + `Alt` + `L`, Mac: `Cmd` + `Option` + `L`
Code › Reformat File → Windows `Ctrl` + `Alt` + `Shift` + `L`, Mac: `Cmd` + `Option` + `Shift` + `L`

VSCode

Format Document → Windows: `Shift` + `Alt` + `F`, Mac: `Shift` + `Option` + `F`, Linux: `Ctrl` + `Shift` + `I`

Eclipse

Code Formatter → Windows: `Ctrl` + `Shift` + `F`, Mac: `Cmd` + `Shift` + `F`.

Sie müssen im ganzen Team dieselben Einstellungen bezüglich Tabulatoren und Leerzeichen haben. Wenn nach dem Einsatz des Formatters alle Zeilen als "verändert"

> angezeigt werden, müssen Sie zuerst diese Einstellungen kontrollieren.
> Nutzen Sie `.editorconfig` um im Projekt einheitliche Formatierungseinstellungen zu konfigurieren.

Verzichten Sie auf veraltete Schnittstellen

Java existiert seit 1995. Seither sind etliche Technologien und Schnittstellen dazu gekommen, die teilweise vorhandene Schnittstellen ersetzen. Um die Rückwärtskompatibilität zu gewährleisten, wurden die Schnittstellen, die ersetzt wurden, oftmals nicht entfernt. Das heisst aber nicht, dass man Sie weiterhin verwenden soll. Achten Sie darauf, insbesondere folgende Schnittstellen nicht mehr zu verwenden:

- `java.util.Date` (Ersatz: `java.time`)
- `java.util.Calendar` (Ersatz: `java.time`)
- `java.net.HttpURLConnection` (Ersatz: `java.net.http`)

Sinnvolle Namensgebung (L1)

Achten Sie bei der Benennung von Variablen, Klassen und Methoden darauf, möglichst beschreibende Namen zu verwenden. Dies hilft, Ihren Code nachzuvollziehen.

Unklare Variablen- und Methodennamen ☹

```
public class Temperature {
    public static BigDecimal calculate(BigDecimal value) { ① ②
        return value.add(273.15);
    }
}
```

① Was macht `calculate` genau?
② Was muss man bei `value` angeben? Celsius? Fahrenheit?

Methoden- und Variablennamen kommunizieren Zweck ☺

```
public class Temperature {
    public static BigDecimal toCelsius(BigDecimal degreesKelvin) { ① ②
        return degreesKelvin.add(273.15);
    }
}
```

① Mit dem Methodennamen `toCelsius` wird kommuniziert, dass der übergebene Wert in Grad Celsius konvertiert wird.
② Dank des Variablennamens `degreesKelvin` weiss der Programmierer, dass die Gradangabe in Kelvin zu erfolgen hat.

Verzichten Sie ausserdem auf die Einbettung von Typen in Variablennamen, Klassen oder Interfaces ("ungarische Notation").

Ungarische Notation ☹

```
public class MathClass { ①
    private static final double S_PI = 3.141; ②

    public static double circleArea(double dRadius) { ③
        return S_PI * dRadius * dRadius;
    }
}
```

① Aufgrund des Suffix `Class` wissen Sie, dass es sich bei `MathClass` um eine Klasse handelt.

② Das Präfix `S_` kommuniziert, dass es sich um eine statische Variable handelt.
③ Das `d` signalisiert, dass in `dRadius` ein `double` gespeichert ist.

Methodengestaltung (L1,L3)

Methoden sind maximal 40 Zeilen lang (L1)

Beschränken Sie die Länge einer Methode auf maximal 40 Zeilen. Reichen die 40 Zeilen nicht aus für die Logik, die Sie unterbringen wollen, machen Sie nicht einfach die Zeilen länger. Lagern Sie stattdessen sinnvolle Teile der Logik (z.B. Bedingungen, Teilberechnungen) in weitere Methoden aus.

Lange Parameterlisten durch Builder oder Argumentobjekt ersetzen (L3) ➕

Vermeiden Sie lange Parameterlisten bei Ihren Methoden, da sie den Code schwer wart- und lesbar machen. Ausserdem wird der Code schwer zu benutzen, da Sie nicht mehr leicht ableiten können, welchen Effekt verschiedene Angaben haben. Beschränken Sie die Anzahl der Parameter auf 4 bis 5 (Faustregel).

Methode mit langer Argumentliste ☹

```
public class BookSearch {

    public List<Book> searchBooks(String title, String author, String isbn, int year,
                                  String publisher, Set<Category> categories) {
        // search catalogue
    }
}
```

Ersetzen Sie lange Parameterlisten durch ein Argumentobjekt.

Argumentobjekt für Suchkriterien ersetzt lange Parameterliste ☺

```
public class BookSearchCriteria {
    private String title;
    private String author;
    private String isbn;

    // etc.

    public String getTitle() {
        return this.title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    // etc.
}

public class BookSearch {

    public List<Book> searchBooks(BookSearchCriteria criteria) {
        // search catalogue
    }
}
```

Wenn Sie den Code noch ein wenig benutzerfreundlicher machen wollen, verwandeln Sie `BookSearchCriteria` in einen Builder.

Builder zur Erstellung von Argumentobjekt ☺

```
public class BookSearchCriteria {
    private final String title; ①
    private final String author;

    private BookSearchCriteria(String title, String author) { ②
        this.title = title;
        this.author = author;
    }

    public String getTitle() {
        return this.title;
    }

    public static class Builder {
        private String title; ③
        private String author;

        public Builder setTitle(String title) {
            this.title = title;
            return this;
        }

        public Builder setAuthor(String author) {
            this.author = author;
            return this;
        }

        public BookSearchCriteria build() {
            return new BookSearchCriteria(this.title, this.author); ④
        }
    }
}
```

① Deklarieren Sie die Variablen `final`, damit die Suchkriterien nach Erstellung des Objekts unveränderlich sind.
② Setzen Sie den Konstruktor `private`, damit der Builder nicht umgangen werden kann.
③ Wenn sinnvoll, setzen Sie hier Standardwerte für die Suchkriterien.
④ Validieren Sie bei Bedarf die Argumente vor Erstellung des Argumentobjekts.

Suchkriterien werden mit Builder erstellt ☺

```
BookSearchCriteria criteria = new BookSearchCriteria.Builder()
        .setAuthor("Joshua Bloch")
        .setTitle("Effective Java")
        .build();
```

Weiterführende Informationen

- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 10-16.

Klassengestaltung

Klassenoberfläche ist minimal (L1)

Machen Sie jede Klasse, ihre Eigenschaften und Methoden so wenig zugänglich wie möglich. Dies bedeutet: Können Sie ein Feld oder eine Methode `private` deklarieren, deklarieren Sie sie `private`. Verwenden Sie weniger restriktive Zugangsbeschränkungen (`protected` oder gar `public`) nur, wenn es unbedingt nötig ist.

Instanzfelder sollten nur in Ausnahmefällen `public` sein. Generieren Sie auch nicht für jedes Feld einen Getter oder Setter, nur weil die IDE dies Ihnen anbietet.

All diese Massnahmen reduzieren die Abhängigkeiten zwischen den verschiedenen Klassen und machen die Wartung des Codes einfacher.

Weiterführende Informationen

- Westphal, Ralf; Lieser, Stefan: Clean Code Developer. Information Hiding Principle [Online]. URL: http://clean-code-developer.de/die-grade/gelber-grad/#Information_Hiding_Principle [Stand: 05.03.2018]

Sinnvolle Abstraktionsebene (L1)

Interfaces statt abstrakte Klassen

Ziehen Sie Interfaces abstrakten Klassen vor (Composition over Inheritance). Abstrakte Klassen geben ein enges Korsett vor: Abgeleitete Klassen müssen eine strikte Hierarchie bilden (Ist-ein-Beziehung) und das Liskovsche Substitutionsprinzip einhalten. Ausserdem kann in Java jeweils nur von einer Klasse geerbt werden. Interfaces sind im Vergleich viel flexibler: Existierende Klassen können jederzeit zusätzliche Interfaces implementieren und nichthierarchische Typsysteme bilden. Ausserdem können existierende Klassen nachträglich mit zusätzlicher Funktionalität (sogenannten Mixins) ausgerüstet werden, ohne die Klassen selber anpassen zu müssen.

Keine unnötige Abstraktion

Verzichten Sie darauf, auf Vorrat abstrakte Klassen und Methoden sowie Interfaces zu erstellen für den Fall, dass man sie in Zukunft *vielleicht* einmal brauchen könnte. Existiert für ein Interface oder eine abstrakte Klasse nur eine einzelne Implementierung, verzichten Sie darauf. Gleiches gilt für ungenutzte Methoden oder Klassen, die fast keine Aufgaben haben.

Weiterführende Informationen

- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 99-103.
- Westphal, Ralf; Lieser, Stefan: Clean Code Developer. Liskov Substitution Principle [Online]. URL: http://clean-code-developer.de/die-grade/gelber-grad/#Liskov_Substitution_Principle [Stand: 05.04.2018]

Klassen stehen für sich allein (L2)

Achten Sie darauf, dass Ihre Klassen für sich alleine stehen. Symptome, dass Ihre Klassen nicht für sich alleine stehen:

- Eine Klasse und ihre Methoden müssen andauernd andere Klassen nach ihrem Zustand fragen.
- Eine Klasse und ihre Methoden müssen mehrere Methoden einer anderen Klasse aufrufen, um etwas zu erledigen.
- Eine Klasse und ihre Methoden müssen Methoden einer anderen Klasse in einer bestimmten Reihenfolge aufrufen.
- Eine Klasse und ihre Methoden müssen über mehrere Objekte hinweg nach Daten fragen (z.B.

```
this.getOther().getAnother().getData())
```

In diesem Fall ist es angebracht, die Funktionalität in die anderen Klassen zu verschieben respektive von dort in die aktuelle Klasse zu bewegen, um die Abhängigkeiten zu reduzieren.

Weiterführende Informationen

- Westphal, Ralf; Lieser, Stefan: Clean Code Developer. Tell, don't ask [Online]. URL: http://clean-code-developer.de/die-grade/gruener-grad/#Tell_dont_ask [Stand: 05.03.2018]
- Westphal, Ralf; Lieser, Stefan: Clean Code Developer. Law of Demeter [Online]. URL: http://clean-code-developer.de/die-grade/gruener-grad/#Law_of_Demeter [Stand: 05.03.2018]

Testing

Ein sinnvoller Unit-Test pro Klasse ist vorhanden (L1)

Zu jeder Klasse in Ihrem Projekt sollte sich ein sinnvoller Unit Test befinden, der ein Mindestmass an Logik (= ein Ausdruck, der etwas berechnet) testet (= mindestens eine Assertion ist vorhanden).

Ein sinnvoller Test ☺

```
class EmployeeTest {

    @Test
    void fullNameComposedOfFirstAndLastName() {
        Employee employee = new Employee();
        employee.setName("Diana");
        employee.setLastName("Prince");
        assertEquals("Diana Prince", employee.getFullName()); ①
    }
}
```

① Mindestens eine Assertion, die mehr als `assertTrue(true)` testet, ist gefordert!

Logik mit Tests abgedeckt (L1)

Die wesentliche Logik Ihrer Software ist mit automatisierten Tests abgedeckt. "Wesentlich" bedeutet, dass Sie weder Getter noch Setter testen brauchen, solange sie nichts anderes machen, als Felder zu lesen beziehungsweise zu setzen. Dasselbe gilt für Hilfsmethoden wie `toString()`. Bei allem anderen sollte die korrekte Funktionsweise durch einen automatisierten Test mit geeigneten Assertions sichergestellt sein.

Kollaborateure als Konstruktor-Argumente (L2)

Wenn Ihre Klasse (nennen wir sie `A`) eine andere Klasse (`B`) verwendet, um eine Aufgabe zu erledigen, ist `B` ein Kollaborateur oder eine Abhängigkeit (Dependency) von `A`. Übergeben Sie `B` immer als Konstruktor-Argument an `A`, statt `B` im Konstruktor von `A` mit `new` zu erstellen. Das macht es einfacher, die Klasse zu testen.

Abhängigkeiten werden im Konstruktor erstellt ☹

```
public class SomeEntityService {
    private final SomeEntityDao someEntityDao;

    public SomeEntityService() {
        this.someEntityDao = new SomeEntityDao();
    }
}
```

Wenn Sie in einem Test `SomeEntityDao` durch einen Mock ersetzen wollen, sind Sie aufgeschmissen. Übergeben Sie stattdessen `SomeEntityDao` als Konstruktorargument und Sie können im Rahmen eines Tests eine andere Instanz übergeben.

Abhängigkeiten werden als Konstruktorargument übergeben ☺

```
public class SomeEntityService {
    private final SomeEntityDao someEntityDao;

    public SomeEntityService(SomeEntityDao dao) { ①
        this.someEntityDao = dao;
    }
}
```

① Potenzielle Verbesserung: Extrahieren Sie ein Interface aus `SomeEntityDao`, dann brauchen Sie nicht

einmal eine Mocking-Bibliothek, um einen Mock zu erstellen.

Unfreundlich zum Testen sind auch statische Methoden, da Sie diese ebenfalls nicht ersetzen können:

Statische Methoden sind untestbar ☹

```
public class SomeEntityService {
    private final SomeEntityDao someEntityDao;

    public SomeEntityService(SomeEntityDao dao) {
        this.someEntityDao = dao;
    }

    public void createSomeEntity(String name) {
        entity = new SomeEntity();
        entity.setName(name);
        entity.dateCreated(Instant.now()) ①
        this.someEntityDao.save(entity);
    }
}
```

① In einem Test wissen Sie nicht, welche Zeit gesetzt wird. Sie können sich in Assertions bestenfalls annähern und einen Hack wie `assertTrue(dateCreated.isAfter(Instant.now().minusSeconds(2)))` in Kombination mit `assertTrue(dateCreated.isBefore(Instant.now()))` verwenden.

Abhilfe schafft, keine statischen Methoden zu verwenden oder sie mit Argumenten zu modifizieren. `java.time` bietet eine elegante Möglichkeit mit der abstrakten `Clock`, für die verschiedene Implementierungen existieren — unter anderem die Systemuhr, die jeweils die aktuelle Zeit zurückgibt, und eine Variante, die immer dieselbe vorgegebene Zeit zurückgibt (`Clock.fixed(Instant.now(), ZoneOffset.UTC)`) und damit ideal für Tests ist.

Parameterisierte statische Methode erhöht Testbarkeit ☺

```
public class SomeEntityService {
    private final SomeEntityDao someEntityDao;

    public SomeEntityService(SomeEntityDao dao) { ①
        this.someEntityDao = dao;
    }

    public void createSomeEntity(String name, Clock clock) {
        entity = new SomeEntity();
        entity.setName(name);
        entity.dateCreated(Instant.now(clock)) ②
        this.someEntityDao.save(entity);
    }
}
```

① Übergeben Sie `Clock` als Konstruktorargument. Dies erlaubt Ihnen später, die Uhr für Tests auszutauschen.

② Übergeben Sie die `Clock` zur Bestimmung der aktuellen Zeit.

Weiterführende Informationen

- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 20-21.
- Westphal, Ralf; Lieser, Stefan: Clean Code Developer. Dependency Inversion Principle [Online]. URL: http://clean-code-developer.de/die-grade/gelber-grad/#Dependency_Inversion_Principle [Stand: 05.03.2018]

Parametrisieren von Tests (L3) ➕

Clean Code Regeln fordern, dass man Wiederholungen vermeidet. Betrachten wir folgendes Beispiel:

```
class DistanceConversionTest {
    @Test
    void testConversionRoundTrip() {
        assertRoundTrip(1);
        assertRoundTrip(1_000);
        assertRoundTrip(9_999_999);
    }
    private void assertRoundTrip(int kilometers) {
        Distance expectedDistance = new Distance(DistanceUnit.KILOMETERS, kilometers);
        Distance actualDistance = expectedDistance
            .convertTo(DistanceUnit.MILES)
            .convertTo(DistanceUnit.KILOMETERS);
        Assertions.assertEquals(expectedDistance, actualDistance);
    }
}
```

Bei einem Fehler in dem ersten Assert stoppt schon den ganzen Test. Besser wäre es mit Parametern:

```
class DistanceConversionTest {
    @ParameterizedTest(name = "#{index}: {0}km == {0}km->mi->km")
    @ValueSource(ints = {1, 1_000, 9_999_999})
    void testConversionRoundTrip(int kilometers) {
        Distance expectedDistance = new Distance(DistanceUnit.KILOMETERS, kilometers);
        Distance actualDistance = expectedDistance
            .convertTo(DistanceUnit.MILES)
            .convertTo(DistanceUnit.KILOMETERS);
        Assertions.assertEquals(expectedDistance, actualDistance);
    }
}
```

Mock-Objekte statt externe Kollaborateure (L3)

Komponenten (wie ein Data Access Object) verwenden in der Regel andere Komponenten (wie eine Datenbankverbindung), um eine Aufgabe zu erledigen. Um eine Komponente sinnvoll zu testen, ist es ratsam, diese von den anderen Komponenten zu isolieren. In Kollaborateure als Konstruktor-Argumente (L2) wird der erste Schritt erklärt, um eine Komponente zu isolieren, nämlich die Übergabe von Kollaborateuren als Konstruktorargument. Der zweite Schritt ist, anstelle des eigentlichen Kollaborateurs eine Testattrappe oder Mock-Objekt zu übergeben. Mit Hilfe einer Testattrappe können Sie in einem Test verifizieren, wie die zu testende Komponente mit dem eigentlichen Kollaborateur interagieren würde und ob sie es richtig tut.

Zu testen: Wird die Buchbewertung korrekt hinzugefügt?

```
class BookService {
    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void reviewBook(ISBN isbn, Customer reviewer, String review) {
        Book book = this.bookRepository.getBook(isbn);
        book.addReview(reviewer, review);
    }
}
```

```java
class BookRepository {
    private final DataSource dataSource;

    public BookRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public Book getBook(ISBN isbn) {
        // call database, return book
    }
}
```

Wie lässt sich testen, ob die Bewertung zum richtigen Buch hinzugefügt würde, ohne die Datenbank zu bemühen? Indem man das `BookRepository` durch eine Testattrappe ersetzt, die ohne Datenbank auskommt.

Test des `BookService` *in Isolation* ☺

```java
class BookServiceTest {

    @Test
    void reviewAdded() {
        Book testBook = new Book();
        TestBookRepository bookRepository = new TestBookRepository(testBook);
        BookService bookService = new BookService(bookRepository);

        bookService.reviewBook(new ISBN("123"), new Customer(), "Great book");

        assertEquals(bookRepository.getLastUsedIsbn(), new ISBN("123"));
        assertEquals(1, testBook.getReviews().size());
    }

    class TestBookRepository extends BookRepository { ①
        private final Book book;
        private ISBN lastUsedIsbn;

        TestBookRepository(Book book) { ②
            this.book = book;
        }

        @Override
        public Book getBook(ISBN isbn) { ③
            this.lastUsedIsbn = isbn; ④
            return book;
        }

        ISBN getLastUsedIsbn() {
            return lastUsedIsbn;
        }
    }
}
```

① Indem `TestBookRepository` von `BookRepository` erbt, kann es anstelle des `BookRepository`, das eine Datenbankverbindung bräuchte, an den `BookService` übergeben werden.

② Die Methode `Book getBook(ISBN)` muss ein Buch zurückgeben. Dieses wird über den Konstruktor des `TestBookRepository` für die spätere Rückgabe bereitgelegt.

③ Durch das Überschreiben der Methode `Book getBook(ISBN)` wird die Datenbankverbindung der ursprünglichen Implementierung nicht mehr benötigt. Die restliche Logik in der ursprünglichen Implementierung kann ebenfalls ignoriert werden, da man sich nur noch für die Interaktion von `BookService` mit `BookRepository` interessiert: Wird die richtige ISBN übergeben und das Buch korrekt weiterverwendet?

④ Damit man überprüfen kann, ob die richtige ISBN übergeben wurde, muss das `TestBookRepository` sie sich merken und über einen separaten Getter zugänglich machen.

> Die ist eine Vorarbeit für die Benutzung von Mock-Libraries. Sie vereinfachen das Schreiben von komplexen Tests.

Mockito (L4) ➕

Sie können auch relativ elegant Objekten-stubs mit Mockito erstellen.

```
import static org.mockito.Mockito.*;

// mock creation
List mockedList = mock(List.class);

// using mock object - it does not throw any "unexpected interaction" exception
mockedList.add("one");
mockedList.clear();

// selective, explicit, highly readable verification
verify(mockedList).add("one");
verify(mockedList).clear();
```

Und auch Funktionen-Aufrufe abfangen, mocken und zählen:

```
// you can mock concrete classes, not only interfaces
LinkedList mockedList = mock(LinkedList.class);

// stubbing appears before the actual execution
when(mockedList.get(0)).thenReturn("first");

// the following prints "first"
System.out.println(mockedList.get(0));

// the following prints "null" because get(999) was not stubbed
System.out.println(mockedList.get(999));


someCodeThatInteractsWithMock();

verify(mock, times(3)).clear();
verify(mock, atLeastOnce()).add(anyObject());
```

Somit können sie vordefiniert und nicht zugreifbarer Code testen und simulieren. Sie können sogar Veränderungen simulieren (e.g. externes Code noch nicht verfügbar, Verhalten aber schon definiert). Das erlaubt Tests schneller zu entwickeln und gründlicher zu gestalten.

Wartbarkeit

Vernünftiger Einsatz von Feldern (L1)

Speichern Sie nur in Feldern, was sie für einen längeren Zeitraum (= länger als der aktuelle Methodenaufruf) brauchen. Für alles andere reicht eine lokale Variable.

Interfaces statt konkrete Typen (L1)

Beziehen Sie sich beim Referenzieren von Parametern, Rückgabewerten, Variablen und Feldern auf Interfaces (wie `List`) anstatt auf konkrete Typen (wie `ArrayList`), wenn ein geeignetes Interface existiert. Verwenden Sie konkrete Typen nur beim Erstellen des Typs. Dies macht Ihr Programm viel flexibler. Wenn Sie feststellen, dass eine andere Implementierung eines Interfaces geeigneter ist, brauchen Sie nur den Konstruktoraufruf auszutauschen und Ihr Programm funktioniert weiter.

Interface wird anstelle des konkreten Typs verwendet ☺

```
SortedSet<String> aSet = new TreeSet<String>();
```

Wenn Sie mehrere Interfaces zur Auswahl haben, verwenden Sie dasjenige, das Ihnen die Funktionalität bietet, die Sie benötigen.

> *Achtung vor versteckten Anforderungen*
>
> Achten Sie darauf, das Interface spezifisch genug zu wählen, um nicht über versteckte Anforderungen zu stolpern. Folgende Deklaration könnte gefährlich werden:
>
> ```
> Set<String> aSet = new TreeSet<String>();
> ```
>
> Wenn Sie ein `TreeSet` verwenden, werden die im Set gespeicherten Strings automatisch sortiert, selbst wenn Sie das Set nur über das Interface `Set` ansprechen. Falls Sie eines Tages das `TreeSet` aus Performance-Gründen in ein `HashSet` verwandeln, geht die Eigenschaft der automatischen Sortierung verloren. Wenn es für das restliche Programm wichtig ist, dass die Werte sortiert sind, wählen Sie als Interface `SortedSet`, da dieses die Sortierung garantiert. Sonst verzichten Sie am besten gleich auf `TreeSet` und wählen von Beginn weg `HashSet`.

Weiterführende Informationen

- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 280-281.

Angemessene Verwendung von Collections (L1)

Wenn Sie Objekte in Collections (Array, List, ...) ablegen, achten Sie darauf, die geeignete Collection zu wählen. Die Auswahl ist gross und Sie können sich dadurch eine Menge Arbeit sparen. Ein `SortedSet` oder eine `SortedMap` sind zum Beispiel implizit sortiert, bei einer `ArrayList` müssen Sie sich selbst darum kümmern.

- Wenn Sie spezielle Anforderungen haben, studieren Sie die Übersicht über das Java Collections Framework, um eine passende Collection zu finden.
- Verwenden Sie `ArrayList` anstelle von Arrays. `ArrayList` ist typsicher während der Übersetzung, d.h. der Compiler hält Sie davon ab, ungültige Datentypen in einer `ArrayList` zu speichern. Arrays sind dagegen nur zur Laufzeit typsicher. `ArrayList` ist ausserdem flexibler bezüglich Iteration, unterscheidet zwischen belegten Elementen und seiner tatsächlichen Länge und seine Länge kann dynamisch verändert werden.
- Die Hilfsklasse `java.util.Collections` bietet eine grosse Auswahl nützlicher Methoden an, u.a. zum Kopieren und Sortieren von Collections. Wenn Sie Collections mit keinem oder nur einem Element

benötigen, verwenden Sie `Collections.emptyList()` und Konsorten beziehungsweise `Collections.singletonList(T o)` und so weiter. Diese sind effizienter, als wenn Sie selber die entsprechenden Collections erstellen.

- Verzichten Sie auf `java.util.Vector`.

> Studieren Sie das Javadoc der verschiedenen Collections und insbesondere deren Konstruktoren. Eine `LinkedBlockingQueue` kann man zum Beispiel mit den richtigen Konstruktor-Argumenten in ihrer Grösse begrenzen oder aus einer `LinkedHashMap` durch Überschreiben einer Methode ein Cache implementieren.

Klassen, ENUMs statt primitive Typen oder Strings (L2)

Verwenden Sie Klassen und ENUMs anstelle von primitiven Typen und Strings. Denn solange Sie Klassen oder ENUMs verwenden, hilft Ihnen der Compiler dabei, keine Tippfehler zu machen oder Wertebereiche zu beschränken. Verwenden Sie dagegen primitive Typen oder Strings, sind Sie auf sich allein gestellt.

String zum Typenvergleich ☹

```
public class Card {

    private final String type = "DIAMOND";

    public void display() {
        switch(type) {
            case "DIAMOND": ①
                System.out.println("♦");
                break;
            // etc.
        }
    }
}
```

① Damit der Vergleich funktioniert, müssen Sie immer exakt `DIAMOND` schreiben, nicht `diamond` oder eine andere Variante.

Typsichere Vergleiche dank ENUM ☺

```
enum CardType {
    DIAMOND, CLUB, HEART, SPADE
}

public class Card {

    private final CardType type = CardType.DIAMOND;

    public void display() {
        switch(type) {
            case DIAMOND:
                System.out.println("♦");
                break;
            // etc.
        }
    }
}
```

> Wenn Sie tatsächlich einmal nicht darum herumkommen, einen ENUM-Wert als String entgegenzunehmen, konvertieren Sie ihn frühzeitig zurück zum ENUM mittels `valueOf(String)`, z.B. `CardType.valueOf("DIAMOND")`.

Verwenden Sie ausserdem Klassen, um komplexe Konzepte zu modellieren, die z.B. aus Wert plus Einheit bestehen (Geld, Temperatur, ...), aus mehreren Werten (Adresse, ...) oder zu denen spezielle (Formatierungs-)Regeln gehören (wie bei Telefonnummern).

Produktpreis modelliert durch primitiven Typ ☹

```
public class Product {
    private double price; ① ②
}
```

① Wenn eine exakte Repräsentation gefragt ist, verwenden Sie nie einen `double`, sondern einen `BigDecimal`.

② Dem `double` mangelt es an Ausdrucksstärke: Weder kann er eine Einheit wie CHF transportieren noch sich selber validieren.

Produktpreis modelliert durch eine Klasse ☺

```
public class Product {
    private Price price;
}

public class Price {
    private BigDecimal amount;
    private Currency currency;
}

private Currency {
    private String name; // like CHF
}
```

Ausnahmen werden behandelt (L2) ➕

Treffen Sie in Ihrem Programm auf eine Ausnahmesituation, werfen Sie eine Exception, anstatt das Problem zu ignorieren und auf das Beste zu hoffen.

Unerwartete Situation löst Exception aus ☺

```
File someFile = new File(path);
if (!someFile.canRead()) {
    String message = String.format("File '%s' is not readable", path); ①
    throw new FileNotReadableException(message);
}
// Read file
```

① Diagnoseinformation, die mitteilt, welche Datei nicht lesbar ist.

Um eine eigene Exception zu definieren, erben Sie von der passenden Superklasse `Exception`(**checked**) oder `RuntimeException` (**unchecked**) und überschreiben Sie die Konstruktoren. Informationen zu den Exceptionstypen finden Sie auch unter Unchecked Exceptions — The Controversy. Achten Sie in jedem Fall darauf, Diagnoseinformationen mitzugeben. Das erleichtert Ihnen die Fehlerdiagnose und Behebung.

Selbst definierte Exception ☺

```
public class FileNotReadableException extends RuntimeException {
    public FileNotReadableException(String message) { ①
        super(message);
    }

    public FileNotReadableException(String message, Throwable cause) { ②
```

```
        super(message, cause);
    }
}
```

① Konstruktor, der eine einzelne Mitteilung akzeptiert.
② Konstruktor, dem Sie zusätzlich zur Mitteilung eine existierende Exception mitgeben können, die die Ursache Ihrer Exception ist (siehe unten).

Das JDK definiert einige Exceptions wie `IllegalArgumentException`. Verwenden Sie diese, wenn sie passen.

Wenn Sie JDK APIs oder Bibliotheken nutzen, kann es sein, dass diese Exceptions werfen. Behandeln Sie diese angemessen:

- Wenn möglich und sinnvoll (was selten der Fall ist), versuchen Sie den Fehler in der Applikation zu korrigieren. Bei einem fehlgeschlagenen Internet-Zugriff können Sie zum Beispiel den Zugriff erneut versuchen.
- Konvertieren Sie die zugrunde liegende Exception in eine geeignete Exception und werfen Sie diese.
- Schreiben Sie eine Meldung ins Log (siehe Logging anstelle von System.out (L3) ➕).

Ignorierte Exception ☹

```
byte[] buffer = new byte[1024];
try (FileInputStream stream = new FileInputStream("/some/file.txt")) {
    stream.read(buffer);
} catch (IOException e) {
    // ignore exception
}
// do something with the buffer's contents
```

Wenn Exceptions ignoriert werden, läuft das Programm weiter und kommt dadurch potenziell in einen inkonsistenten Zustand. In diesem Beispiel weiss man unter anderem nicht, was in der Variable `buffer` gespeichert ist. Zudem erfährt man während des Betriebs nicht, dass es ein Problem gegeben hat.

Exception wird konvertiert und erneut geworfen ☺

```
byte[] buffer = new byte[1024];
try (FileInputStream stream = new FileInputStream("/some/file.txt")) {
    stream.read(buffer);
} catch (IOException e) {
    throw new FileReaderException("Could not read file", e); ①
}
// do something with the buffer's contents
```

① Konvertiert die `IOException` in eine `RuntimeException` und wirft diese erneut.

Wenn Sie eine Exception konvertieren, geben Sie diese an die nächst höhere Exception (in diesem Fall `FileReaderException`) weiter und schlucken Sie sie nicht. Ansonsten geht der Stacktrace verloren und man weiss nicht, was die eigentliche Ursache der `FileReaderException` ist.

Weiterführende Informationen

- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 293-310.

Logging anstelle von System.out (L3) ➕

Geben Sie Debug- und Informationsmeldungen, Warnungen und Fehler mittels Logging aus anstatt `System.out` zu verwenden. Logging ermöglicht, während der Entwicklung und in der Produktion das Log

Level einzustellen. Sie können damit regeln, wie viele Informationen ausgegeben werden sollen. Zudem können Sie steuern, ob die Ausgaben auf die Konsole oder zum Beispiel in eine Datei geschrieben werden sollen. All das geht mit `System.out` respektive `System.err` nicht.

Exception wird direkt auf die Konsole geschrieben ☹

```
public EmployeeService(ServletContext servletContext) {
    try {
        System.out.println("Establishing database connection"); ①
        loadH2Driver();
        establishH2DBConnection(servletContext);
        setUpH2Database();
    } catch (Exception e) {
        e.printStackTrace(); ②
    }
}
```

① Gibt Debug-Meldung auf der Konsole aus.
② Gibt Stacktrace auf der Konsole aus.

Es gibt viele Möglichkeiten, um in Java zu loggen. Java bringt im Package `java.util.logging` ("JUL" genannt) sogar selber die nötigen Werkzeuge mit.

Exception wird geloggt ☺

```
public class EmployeeService {

    private static final Logger logger = Logger.getLogger(EmployeeService.class.
getName()); ①

    public EmployeeService(ServletContext servletContext) {
        try {
            logger.info("Establishing database connection"); ②
            loadH2Driver();
            establishH2DBConnection(servletContext);
            setUpH2Database();
        } catch (Exception e) {
            logger.log(Level.SEVERE, "Could not establish database connection", e); ③
        }
    }
}
```

① Initialisiert den Logger. Achten Sie darauf, den Namen der umgebenden Klasse (`EmployeeService` in diesem Fall) zu verwenden und das Feld `static final` zu deklarieren.
② Gibt Debug-Meldung via Logger aus, wenn das Log Level `INFO` oder tiefer ist.
③ Gibt Fehlermeldung via Logger aus, wenn das Log Level `SEVERE` oder tiefer ist.

Wenn Sie Log-Meldungen aus Variablen zusammenbauen, verwenden Sie keine String-Konkatenation.

String-Konkatenation für Log-Meldungen ☹

```
logger.info("Unknown option " + userOption + ", using default " + defaultOption);
```

Die String-Konkatenation wird immer ausgeführt, selbst wenn die Log-Nachricht gar nicht ausgegeben wird, weil das Log-Level der Meldung im konfigurierten Log-Level der Applikation nicht eingeschlossen ist (z.B. bei `error` wird `info` nicht ausgegeben). Nutzen Sie daher Variablen-Interpolation. Die Variablen werden dabei erst eingesetzt, wenn die Nachricht tatsächlich ausgegeben wird.

Variablen-Interpolation für Log-Meldungen ☺

```
logger.log(Level.INFO, "Unknown option {0}, using default {1}", userOption,
```

```
defaultOption);
```

Was durch die Logger wo (Konsole, Datei, ...) ausgegeben wird, können Sie über eine Datei namens `logging.properties` steuern. Wie Sie sie nutzen, wird zum Beispiel im Buch *Java SE 8 Standard-Bibliothek* von Christian Ullenboom (Rheinwerk) im Kapitel Logging mit Java beschrieben.

Anwender erhält sinnvolle Fehlermeldung (L3) ➕

Wenn eine Aktion in Ihrer Anwendung fehlschlägt, ist es nicht nur wichtig, den Fehler zu loggen, sondern auch dem Anwender eine sinnvolle Fehlermeldung anzuzeigen. Achten Sie darauf:

- dass unter keinen Umständen nur eine leere View erscheint.
- dass die Erklärung nicht unverständlich ist ("500 Internal Server Error").
- dass nie der Stacktrace angezeigt wird, da ihn einerseits die Besucher nicht verstehen und andererseits das als Einfallstor für Angreifer dienen kann.
- dass den Benutzern erklärt wird, was sie tun können, um das Problem zu beheben (z.B. zur Sitemap gehen, wenn Seite nicht existiert), sofern das möglich ist.

Javadoc (L1)

Beschreiben Sie in Javadoc den Zweck der von Ihnen erstellten Klassen und Interfaces.

Dokumentieren Sie ausserdem Methoden in Interfaces, abstrakte Methoden sowie öffentliche Methoden von Klassen, die die Rolle von internen Schnittstellen einnehmen wie zum Beispiel Data Access Objects. Für Methoden ist es wichtig, dass der Javadoc-Kommentar den Vertrag zwischen Methode und anwendendem Code beschreibt:

- Es ist erklärt, *was* die Methode macht und nicht *wie* sie es macht.
- Es sind sämtliche Vorbedingungen aufgelistet (z.B. dass Argument nicht `null` sein darf).
- Es sind sämtliche Nachbedingungen aufgelistet (z.B. dass die retournierte Collection nie `null` sein kann).
- Es ist jede Exception aufgelistet, die auftreten kann und in welcher Situation das passiert.
- Jeder Parameter ist beschrieben.
- Sofern sinnvoll ist der Rückgabewert beschrieben.

Für abstrakte Methoden ist ausserdem wichtig, dass Sie beschreiben, welche Erwartungen an eine Implementierung gestellt werden.

Javadoc für eine abstrakte Methode aus Spring ☺

```
public abstract class AbstractBeanFactory extends FactoryBeanRegistrySupport implements
ConfigurableBeanFactory {
    /**
     * Return the bean definition for the given bean name. ①
     *
     * Subclasses should normally implement caching, as this method is invoked ②
     * by this class every time bean definition metadata is needed.
     *
     * <p>Depending on the nature of the concrete bean factory implementation,
     * this operation might be expensive (for example, because of directory lookups
     * in external registries). However, for listable bean factories, this usually
     * just amounts to a local hash lookup: The operation is therefore part of the
     * public interface there. The same implementation can serve for both this
     * template method and the public interface method in that case.
     *
```

```
     * @param beanName the name of the bean to find a definition for ③
     * @return the BeanDefinition for this prototype name (never {@code null}) ④
     * @throws org.springframework.beans.factory.NoSuchBeanDefinitionException ⑤
     * if the bean definition cannot be resolved
     * @throws BeansException in case of errors
     *
     * @see RootBeanDefinition
     * @see ChildBeanDefinition
     * @see
org.springframework.beans.factory.config.ConfigurableListableBeanFactory#getBeanDefinitio
n
     */
}
```

① Kurzbeschrieb der Methode. Erscheint in der Methodenübersicht im Javadoc.
② Anweisungen für die Implementierung.
③ Beschreibung der Parameter
④ Beschreibung des Rückgabewerts inklusive Nachbedingungen
⑤ Beschreibung der möglicherweise auftretenden Exceptions inklusive Erklärung, wann sie auftreten können.

Weiterführende Informationen

- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 254-260.

Dokumentieren mit Beispielen (L2) ➕

Trotz Clean Code gibt es keine Garantie, dass man eine Funktion auf den ersten Blick sofort versteht. Beispiele mit konkreten Funktionsaufrufe erleichtern und beschleunigen die Entwicklung in Teams. Es gibt aber keinen Zwang bei jeder Funktion dies zu tun (insbesondere für Getter und Setter)! Am unteren Beispiel gibt es einen relativen schwer lesbaren regulären Ausdruck.

```
class Supply {
    /**
    * The code universally identifies a supply.
    *
    * It follows a strict format, beginning with an S (for supply), followed
    * by a five digit inventory number. Next comes a backslash that
    * separates the country code from the preceding inventory number. This
    * country code must be exactly two capital letters standing for one of
    * the participating nations (US, EU, RU, CN). After that follows a dot
    * and the actual name of the supply in lowercase letters.
    */
    static final Pattern code = Pattern.compile("^S\\d{5}\\\\(US|EU|RU|CN)\\.[a-z]+$");
}
```

Durch Beispiele ist es viel einfacher zu verstehen, wann es zutrifft. Nebenbemerkung: man kann diese Beispiele direkt in den Tests verwenden.

```
class Supply {
    /**
    * The expression universally identifies a supply code.
    *
    * Format: "S<inventory-number>\<COUNTRY-CODE>.<name>"
    *
    * Valid examples: "S12345\US.pasta", "S08342\CN.wrench",
    * "S88888\EU.laptop", "S12233\RU.brush"
    *
```

```
    * Invalid examples:
    * "R12345\RU.fuel" (Resource, not supply)
    * "S1234\US.light" (Need five digits)
    * "S01234\AI.coconut" (Wrong country code. Use US, EU, RU, or CN)
    * " S88888\EU.laptop " (Trailing whitespaces)
    */
    static final Pattern supplyCode =
            Pattern.compile("^S\\d{5}\\\\(US|EU|RU|CN)\\.[a-z]+$");
}
```

Korrektheit

Object-Klassenvertrag wird eingehalten (L1)

Vermeiden Sie nach Möglichkeit, die Methoden `equals(Object)` und `hashCode()` der Klasse `Object` zu überschreiben. Falls Sie es doch tun, beachten Sie folgende Kriterien:

- Die Klasse muss ein Value Type, also unveränderlich, sein.
- Wenn `equals(Object)` überschrieben wird, muss gleichzeitig auch `hashCode()` überschrieben werden und umgekehrt.
- Der Klassenvertrag, also die Javadoc-Beschreibung, muss unbedingt eingehalten werden.

Halten Sie sich unbedingt an diese Regeln, da es ansonsten zu überraschenden Fehlern kommen kann, beispielsweise dass Objekte in sortierten Collections nicht am richtigen Ort erscheinen oder nicht gefunden werden. Wie Sie die Anforderungen richtig umsetzen ist detailliert im Buch *Effective Java* beschrieben (siehe Referenzen).

Weiterführende Informationen

- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 37-54.

Eingabeüberprüfung (L1) ➕

Argumentüberprüfung

Überprüfen Sie Argumente, bevor Sie sie weiterverarbeiten. Zum Beispiel dürfen Index-Angaben für Arrays nicht negativ sein oder eine Division durch Null ist nicht erlaubt. Verhindern Sie diese Probleme nicht frühzeitig, können unverständliche Exceptions auftreten oder stillschweigend ungültige Werte berechnet werden. Indem Sie die Argumente frühzeitig validieren und Exceptions werfen, können Sie Ihren Code robuster gestalten, Sicherheitsprobleme und Exceptions verhindern.

Modulus wird frühzeitig validiert (Ausschnitt aus JDK 8) ☺

```
/**
 * Returns a BigInteger whose value is {@code (this mod m}).  This method
 * differs from {@code remainder} in that it always returns a
 * <i>non-negative</i> BigInteger.
 *
 * @param  m the modulus.
 * @return {@code this mod m}
 * @throws ArithmeticException {@code m} &le; 0 ③
 * @see    #remainder
 */
public BigInteger mod(BigInteger m) {
    if (m.signum <= 0) { ①
        throw new ArithmeticException("BigInteger: modulus not positive"); ②
    }
    // Actual computation
}
```

① Bevor die eigentliche Berechnung ausgeführt wird, wird geprüft, ob der Modulus negativ ist.

② Es wird eine Exception geworfen, bevor der Code während der Berechnung in einen möglicherweise ungültigen Zustand kommt.

③ Dokumentieren Sie die Exception und in welchen Fällen sie geworfen wird im Javadoc!

Man kann sich streiten, ob vor dem Zugriff auf `m.signum` geprüft werden sollte, ob `m` nicht eventuell `null` ist. Weil `m` direkt verwendet wird, kann man sich das sparen, weil man weiss, dass Java eine `NullPointerException` werfen wird. Würde `m` dagegen erst später verwendet werden, wäre dies dagegen nötig, wie im folgenden Konstruktor:

Argumentüberprüfung warnt frühzeitig vor ungültigem Argument ☺

```
public class LogBook {

    private final Path location;

    public SomeEntityDao(Path location) {
        Objects.requireNonNull(location, "Location may not be null"); ①
        this.location = location;
    }

    public List<Files> getAll() {
        try(Stream<Path> paths = Files.walk(this.location, 2)) { ②
            paths.filter(Files::isRegularFile)
                    .forEach(System.out::println);
        }
    }
}
```

① Ist `location` `null`, wird eine `NullPointerException` geworfen.

② Würde `location` nicht im Konstruktor validiert, würde die `NullPointerException` erst hier auftreten und das könnte auch zeitlich viel später sein. Je nach Architektur Ihrer Software wird die fehlende `DataSource` mit der Validierung schon beim Applikationsstart bemerkt und nicht erst, wenn `getAll()` verwendet wird.

Weiterführende Informationen

- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 227-230.
- Munroe, Randall: XKCD, Exploits of a Mom [Online]. URL: https://www.xkcd.com/327/ [Stand: 05.03.2018]
- Open Web Application Security Project: SQL Injection Prevention Cheat Sheet [Online]. URL: https://www.owasp.org/index.php/SQL_Injection_Prevention_Cheat_Sheet [Stand: 05.03.2018]

Null-(Leer-)Objekt anstelle von null als Rückgabewert (L2)

Geben Sie als Rückgabewerte von Methoden nach Möglichkeit ein sogenanntes Null- bzw. Leer-Objekt zurück und nicht `null` an sich. Damit werden unabsichtliche `NullPointerException` verhindert.

Grosse Gefahr für `NullPointerException` *im aufrufenden Code* ☹

```
public class MenuPlan {

    private List<Menu> menus;

    public List<String> getMenuNames() {
        if (menus == null) {
            return null;
        }
        return menus.stream().map(Menu::getName).collect(Collectors.toList());
    }
}
```

Gefahrlose Methodennutzung dank Null- bzw. Leer-Objekt ☺

```
class MenuPlan {

    private List<Menu> menus;

    public List<String> getMenuNames() {
        if (menus == null) {
```

```
            return Collections.emptyList();
        }
        return menus.stream().map(Menu::getName).collect(Collectors.toList());
    }
}
```

Obige Lösung kann noch verbessert werden, indem Sie verunmöglichen, dass das Feld `menus` jemals `null` sein kann.

Optional-Container anstelle von null Rückgabewert (L2) ➕

Geben Sie als Rückgabewerte von Methoden nicht `null` zurück um zu signalisieren, dass kein Wert vorhanden ist, sondern verwenden Sie Optional-Container. Damit verhindern Sie fehleranfällige `NullPointerException`.

Grosse Gefahr für `NullPointerException` im aufrufenden Code ☹

```
public class UserPrincipal {
    private String userId;

    private UserPrincipal(String userId) {
        this.userId = userId;
    }

    public static UserPrincipal userForCredentials(String userId, String password) {
        UserPrincipal loginUser = null;
        if (verifyLogin(userId, password)) {
            loginUser = new UserPrincipal(userId);
        }
        return loginUser; ①
    }
}
```

① Wenn kein User gefunden wurde, bzw. die Benutzerauthentisierung fehlgeschlagen ist, wird `null` zurückgegeben.

Gefahrlose Methodennutzung dank Optional-Container ☺

```
public class UserPrincipal {
    private String userId;

    private UserPrincipal(String userId) {
        this.userId = userId;
    }

    public static Optional<UserPrincipal> userForCredentials(String userId, String
password) {
        if (verifyLogin(userId, password)) {
            return Optional.of(new UserPrincipal(userId)); ①
        } else {
            return Optional.empty(); ②
        }
    }
}
```

① gültiger Rückgabewert wird in den Optional-Container verpackt.
② Wird kein Wert gefunden wird ein "leerer" Container zurückgegeben.

Bei der Auswertung des Rückgabewertes muss dann das Fehlen explizit behandelt werden.

Auswertung des Optional-Rückgabewerts

```
// Variante 1: Unterscheidung des gültigen bzw. ungültigen Wertes.
Optional<UserPrincipal> user = UserPrincipal.userForCredentialsOptional(userId,
password);
if (user.isPresent()) { ①
    System.out.println("Login successful. Welcome " + user.get().getUserId()); ②
} else {
    System.out.println("Login failed or user does not exist");
}

// Variante 2: verwende Defaultwert, wenn Optional leer ist
UserPrincipal user = UserPrincipal.userForCredentialsOptional(userId, password)
    .orElse(new UserPrincipal("anonymous")); ③
```

① Abfrage, ob der Wert im Optional-Container vorhanden ist.

② Auslesen des Wertes aus dem Container. Beim Aufruf von `get()` eines leeren Containers wird eine `NoSuchElementException` geworfen.

③ Variante mit explizitem Default-Wert, falls ein leerer Container zurückgegeben wird.

Ressourcen werden geschlossen (L3) ➕

Schliessen Sie alle Ressourcen (Datenbankverbindungen, Dateien, Streams, ...), die Sie geöffnet haben, sobald Sie sie nicht mehr brauchen. Wenn Sie nicht sicher sind, ob etwas geschlossen werden muss, schauen Sie ins Javadoc des entsprechenden Typs: Wird `java.io.Closeable` oder `java.lang.AutoCloseable` implementiert, handelt es sich um eine schliessbare Ressource und es gibt die Methode `close()`.

Wenn Sie eine Ressource vergessen zu schliessen, kann ein Ressourcen-Leck auftreten. Dies bedeutet, dass Ihr Programm die Ressource besetzt hält, was Speicher und weitere Ressourcen kosten kann (z.B. File Handles) und im schlimmsten Fall andere Programme (oder sogar Ihr eigenes) nicht weiter auf die Ressource zugreifen können, oder dass Ihr Computer den Dienst versagt.

Hier ein Beispiel, wie man Ressourcen korrekt schliessen müsste.

Ressourcen korrekt schliessen 😐

```
public MyData readDate(File dataDir, String dataFileName) throws DataParseException {
    MyData data = new MyData();
    Scanner scanner = null; ①
    try {
        scanner = new Scanner(new File(dataDir, dataFileName));
        while (scanner.hasNextLine()) {
            data.parseLine(scanner.nextLine());
        }
    } catch (IOException e) { ②
        throw new DataParseException(e);
    } finally { ③
        if (scanner != null) { ④
            scanner.close();
        }
    }
    return data;
}
```

① Ressource muss ausserhalb des `try`-Blocks deklarieren, damit im `finally` darauf zugegriffen werden kann.

② Sowohl beim Öffnen der Datei, als auch beim Lesen könnte ein Fehler auftreten, den wir behandeln. Hier wird er als in eine eigene `DataParseException` verpackt, welche im aufrufenden Programm behandelt wird.

③ Damit die Ressource sowohl im Erfolgs-, als auch im Fehlerfall geschlossen wird, muss sie innerhalb von `finally` geschlossen werden, nicht innerhalb oder nach dem `try`-Block.

④ Weil wir nicht wissen, ob das Scanner-Objekt erfolgreich initialisiert wurde, muss es auf `null` überprüft werden.

Dies ist etwas umständlich und kann noch aufwändiger werden, wenn beim Schliessen der Ressourcen Exceptions auftreten können.

Verwenden Sie deshalb nach Möglichkeit Try-With-Resources, das seit Java 8 unterstützt wird, da dann Java für darauf achtet, die Ressource korrekt zu schliessen.

Ressource wird automatisch geschlossen dank Try-With-Resources ☺

```
public MyData readTrack(File dataDir, String dataFileName) throws DataParseException {
    MyData data = new MyData();
    try (Scanner scanner = new Scanner(new File(dataDir, dataFileName))) { ①
        while (scanner.hasNextLine()) {
            data.parseLine(scanner.nextLine());
        }
    } catch (IOException e) { ②
        throw new DataParseException(e);
    }
    return data;
} ③
```

① Die Ressource wird in Klammern des try-Blocks erstellt und ist somit nur innerhalb des try-Blocks gültig. Es können auch mehrere Ressourcen mit Komma separiert erstellt werden, die dann alle automatisch geschlossen werden.

② Fehler müssen immer noch behandelt werden.

③ Beim Verlassen des try-Blocks (egal ob korrekt oder mit Exception) werden alle `Closeable` und `AutoCloseable` Ressourcen automatisch geschlossen.

Falls die Ressource nicht vom Typ `java.io.Closeable` oder `java.lang.AutoClosable` ist, müssen Sie diese wie im ersten Beispiel manuell schliessen.

Beispiel: Schliessen von Ressourcen, die `AutoClosable` nicht implementieren

```
public static void run() {
    TextIO textIO = TextIoFactory.getTextIO();  ①
    try {
        TextTerminal<?> terminal = textIO.getTextTerminal();

        UserPrincipal user = loginUser(textIO);
        if (user == null) {
            terminal.printf("Login failed or user does not exist%n");
            return;
        }
        terminal.printf("Login successful. Welcome %s%n", user.getUserId());
        // continue with application
        showMenu(textIO, user);

    } catch (Exception e) { ②
        textIO.getTextTerminal().printf("An error occurred: %s%n", e.getMessage());
    } finally { ③
        if (textIO != null) { ④
            textIO.dispose("Application has ended.");
        }
    }
}
```

① `org.beryx.textio.TextIO` implementiert `java.lang.Autoclosable` nicht. Es sollte deshalb manuell

sichergestellt werden, dass die Ressource (inkl. Terminal) korrekt geschlossen wird.

② Exceptions in der Anwendung sollen abgefangen und "behandelt" werden.

③ Das textIO-Objekt bzw. das darin erstellte Terminal wird geschlossen.

④ Sicherstellen, dass keine `NullPointerException` erzeugt wird.

Weiterführende Informationen

- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 34-36.

Streams sind seiteneffektfrei (L3) ➕

Das Java Streams API (nicht zu verwechseln mit `OutputStream` und `InputStream` für I/O) ermöglicht, die deklarative Verarbeitung von Daten, wie das in funktionalen Programmiersprachen und SQL der Fall ist. Ein nützlicher Nebeneffekt des Stream API ist, dass die Parallelisierung der Datenverarbeitung sehr einfach möglich und damit Ihr Programm einfach schneller gemacht werden kann. Um all diese Möglichkeiten nutzen zu können, dürfen Sie nicht die klassische objektorientierte (mit Seiteneffekten) mit der funktionalen Programmierung (ohne Seiteneffekte) vermischen.

Folgender Code erstellt eine Liste, welcher Vorname wie häufig vorkommt. Er verwendet einerseits das Stream API (`forEach()`), nutzt aber gleichzeitig das herkömmliche, objektorientierte API.

Seiteneffektbehaftete Nutzung von Streams ☹

```
List<Person> persons = new ArrayList<>();

Map<String, Integer> occurrences = new HashMap<>();
persons.forEach(person ->
        occurrences.merge(person.getFirstName(), 1, Integer::sum)); ①
```

① Aus dem Stream heraus wird die Map `occurrences` modifiziert, die ausserhalb des Streams existiert. Dies ist ein klassischer Seiteneffekt.

Entscheiden Sie sich in diesem Fall entweder für die klassische objektorientierte Variante mit `for` -Schleife oder nutzen Sie das Stream API richtig:

Seiteneffektfreie Nutzung von Streams ☺

```
List<Person> persons = new ArrayList<>();

Map<String, Long> occurrences = persons.stream()
        .collect(groupingBy(Person::getFirstName, counting())); ①
```

① Anstatt `occurrences` aus dem Stream heraus zu modifizieren, liefert der Stream das Endresultat. Dies ist seiteneffektfrei.

Weiterführende Informationen

- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 210-215.

Vermeiden Sie Lecks durch Referenzen (L3) ➕

Wenn Sie eine Klassenattributinstanz als Rückgabewert zurückliefern, dann geben Sie die Referenz des Objekts zurück und es kann von aussen mutiert/verändert werden.

```
public class Inventory {
    private final List<Supply> supplies;
    Inventory(List<Supply> supplies) {
        this.supplies = supplies;
    }
```

```
    List<Supply> getSupplies() {
        return supplies;
    }
    public static void main (String[] args){

        List<Supply> externalSupplies = new ArrayList<>();
        Inventory inventory = new Inventory(externalSupplies);
        System.out.println("Size "+ inventory.getSupplies().size()); // == 0
        externalSupplies.add(new Supply("Apple"));
        System.out.println("Size "+ inventory.getSupplies().size()); // == 1
        inventory.getSupplies().add(new Supply("Banana"));
        System.out.println("Size "+ inventory.getSupplies().size()); // == 2
    }
}
```

In dem obigen Beispiel ist nur die Referenz final nicht der Inhalt der Liste. Besser ist es die ArrayList in eine unmodifiableList zu verwandeln:

```
class Inventory {
    private final List<Supply> supplies;
    Inventory(List<Supply> supplies) {
        this.supplies = new ArrayList<>(supplies);
    }
    List<Supply> getSupplies() {
        return Collections.unmodifiableList(supplies);
    }

    public static void main (String[] args){

        List<Supply> externalSupplies = new ArrayList<>();
        Inventory inventory= new Inventory(externalSupplies);
        inventory.getSupplies().size(); // == 0
        externalSupplies.add(new Supply("Apple"));
        System.out.println("Size "+ inventory.getSupplies().size()); // == 0
        // next line throws UnsupportedOperationException
        System.out.println("Size "+ inventory.getSupplies().add(new Supply("Banana")));
    }
}
```

Varia

Frühe, häufige Commits (L1) ➕

Code, der nur bei Ihnen lokal auf der Festplatte liegt, existiert für Ihre Team-Kollegen nicht. Führen Sie spätestens nach ein paar Stunden Arbeit Ihre Änderungen mit denen der Kollegen zusammen. Je häufiger Sie Änderungen an die Versionskontrolle übergeben, umso kleiner sind ausserdem die Auswirkungen von Datenverlust, ungewollten Änderungen oder Defekten.

Gute Commits (L2) ➕

Tim Pope hat 2008 auf seinem Blog beschrieben, was eine gute Commit-Nachricht *für Git* ausmacht. Und hat dazu diese Modell-Nachricht verfasst:

> Capitalized, short (50 chars or less) summary
>
> More detailed explanatory text, if necessary. Wrap it to about 72 characters or so. In some contexts, the first line is treated as the subject of an email and the rest of the text as the body. The blank line separating the summary from the body is critical (unless you omit the body entirely); tools like rebase can get confused if you run the two together.
>
> Write your commit message in the imperative: "Fix bug" and not "Fixed bug" or "Fixes bug." This convention matches up with commit messages generated by commands like git merge and git revert.
>
> Further paragraphs come after blank lines.
>
> - Bullet points are okay, too
> - Typically a hyphen or asterisk is used for the bullet, followed by a single space, with blank lines in between, but conventions vary here
> - Use a hanging indent
>
> — Tim Pope, A Note About Git Commit Messages

Folgen Sie seinem Rat, und Ihre History wird gleichzeitig informativ und einfach zu verfolgen sein:

Ausschnitt aus der Projekt-History von Spring Boot ☺

```
$ git log --pretty=oneline
bce975c11eb3864ada105937c1233c4701be356a Merge pull request #12066 from
jkschneider:micrometer-428
6c85ed70e44d4523763792c1e8ac9bb2ffa515ef Polish "Add SignalFxConfig source property"
a732053afdfe50467428c05e69afbc3e990d8a99 Add SignalFxConfig source property
fd0293cd5c6c0ee6a494640dc8cd4e272ccbb88c Merge pull request #12106 from
jkschneider:simple-enable
1dab83adaa16ddcf83126f5d52bed3619dd7f6f8 Polish "Restore behavior of
management.metrics.export.simple.enabled"
da759f29d081cca1d8de6e40b88176d147e271b0 Restore behavior of
management.metrics.export.simple.enabled
19ce68d2d8cb6f62aa242ce550c6d7ed6893c321 Prefix endpoint's path to controller endpoint
mappings with no path
da5bebf574cf9f52994b3a960961cde74876999c Revert "Use Jersey BOM in dependency management"
7ace46e7e37abe513e38c6cea599871503daa4d6 Merge pull request #12098 from
sbraconnier:master
1a238bae295a757d086e885fb4e0453596100303 Add JODConverter support
```

```
1d83e87bcdcd0a70680ac834900ddd3821cc65b9 Validate @ConfigurationProperties on @Bean
methods
9e75680e6f26a23494f21fb6c41b70445f58d8ea Polish
d2039128b5878d769f4a36c0d08f0c1e6cec470e Merge branch '1.5.x'
75e82e1b2c4035ca590a8273cead41b58a169776 Fix tests to use GET to bypass CsrfFilter
48d8a4a1178b420d3efeb3aff3ed0df1a81e696a Merge branch '1.5.x'
4e61136948c459dbb37b4d032dba2e397374cb33 Enable CSRF protection by default
db7268b841b9b90d498822b4ab37981792303126 Fix TestRestTemplate request factory management
51de220b555a65e59792cbeed001e94d59fb4787 Enable CSRF protection by default
```

Weiterführende Informationen

- Pope, Tim: A Note About Git Commit Messages [Online]. URL: https://tbaggery.com/2008/04/19/a-note-about-git-commit-messages.html [Stand: 20.02.2018]

Verwenden von Feature-Branches und Pull-Requests (L2) ➕

Arbeiten Sie in einer Gruppe und auf denselben Dateien im Master-Branch, kann es passieren, dass ein Konflikt auftritt, welcher den Arbeitsfluss aller Parteien stört. Arbeitet jeder (Untergruppe) in einem separaten Branch, sinkt die Gefahr. Merging auf dem Master geschieht dann am besten Mittels Pull-request, welches auch eine Diskussion und ggf. Nachbesserung verlangen kann. Dies erhöht die Qualität und schliesslich die Produktivität.

Standard-Java-Verzeichnisstruktur (L2) ➕

Legen Sie Ihre Klassen, Tests und Ressourcen (wie Konfigurationsdateien) in der Standard-Java-Verzeichnisstruktur ab:

src/main/java

Platz für Java-Klassen, jeweils in einem Unterverzeichnis korrespondierend zum Package-Namen (die öffentliche Klasse `Test` im Package `com.example` kommt nach `src/main/java/com/example/Test.java`). Speichern Sie jede Klasse in einer eigenen Datei, egal ob sie öffentlich ist oder nicht.

src/main/resources

Speichern Sie hier Konfigurationsdateien, Bilder oder ähnliches.

src/test/java

Platz für Test-Klassen. Der Test für die öffentliche Klasse `Example` im Package `com.example` kommt nach `src/test/java/com/example/ExampleTest.java`. Es ist wichtig, dass Tests in demselben Package liegen wie die zu testenden Klassen, damit Sie in den Tests auf Methoden und Felder zugreifen können, die package private sind.

src/test/resources

Speichern Sie hier Konfigurationsdateien oder ähnliches für Tests.

Vernünftige Objektlebenszeiten (L2) ➕

Achten Sie insbesondere darauf, aufwendig zu erstellende Objekte wiederzuverwenden und nicht unnötigerweise neu zu kreieren. Das macht Ihr Programm langsamer oder sogar langsam.

Typische Kandidaten:

- Reguläre Ausdrücke
- Formatter wie `DateTimeFormatter`, `NumberFormatter`, `SimpleDateFormat`
- Collections

Ineffiziente Anwendung eines regulären Ausdrucks ☹

```
for (String str : strings) {
    if (str.matches("^[a-z]+$")) { ①
        // do something
    }
}
```

① `str.matches("^$")` ist gleichbedeutend mit `Pattern.compile("^[a-z]$").matcher(str).matches()`, d.h. bei jedem Durchgang wird derselbe reguläre Ausdruck erneut kompiliert. Das ist eine Verschwendung von CPU-Zyklen.

Erstellen Sie reguläre Ausdrücke oder Formatter so wenig wie möglich, indem Sie sie aus Schleifen herausnehmen oder, noch besser, in statische Felder umwandeln.

Effiziente Anwendung eines regulären Ausdrucks ☺

```
public class Example {
    private static final Pattern strPattern = Pattern.compile("^[a-z]+$"); ①

    public void doSomething(List<String> strings) {
        for (String str : strings) {
            if (strPattern.matcher(str).matches()) { ①
                // do something
            }
        }
    }
}
```

① Das ist auch in multi-threaded Anwendungen sicher zu verwenden, weil `Pattern` thread-safe ist. Schauen Sie ins Javadoc der entsprechenden Klassen, wenn Sie unsicher sind, ob eine Klasse thread-safe ist.

Auch mit Collections lässt sich viel Schindluder anstellen. Ein klassisches Eigentor ist, sie immer wieder neu zu erstellen:

Ineffiziente Verwendung einer Collection ☹

```
for (String str : strings) {
    List<SomeEntity> myList = new ArrayList<SomeEntity>();
    // do something with myList
}
```

Verwenden Sie stattdessen die Collection wieder, indem Sie die Methode `clear()` anwenden.

Effiziente Verwendung einer Collection ☺

```
List<SomeEntity> myList = new ArrayList<SomeEntity>(); ①
for (String str : strings) {
    myList.clear(); ②
    // do something with myList
}
```

① Die Liste wird nur noch einmal ausserhalb der Schleife erstellt.

② `clear()` leert die Liste und sie lässt sich wiederverwenden, als wäre sie "neu".

Verwandeln Sie Collections, in die Sie schreiben, nur in Ausnahmefällen in statische Felder, selbst wenn sie thread-safe sind. Die Synchronisation mehrerer schreibender Threads ist aufwendiger und fehleranfälliger als die Erstellung mehrerer Collections.

Funktional über Prozedural

Favor Format over concatenation (L2) ➕

Bei der Ausgabe von Informationen ist die Vermischung von Daten und Layout nicht immer vorteilhaft, insbesondere wegen der Lesbarkeit.

```
class Mission {
    Logbook logbook;
    LocalDate start;
    void update(String author, String message) {
        LocalDate today = LocalDate.now();
        String month = String.valueOf(today.getMonthValue());
        String formattedMonth = month.length() < 2 ? "0" + month : month;
        String entry = author.toUpperCase() + ": [" + formattedMonth + "-" +
                    today.getDayOfMonth() + "-" + today.getYear() + "](Day " +
                    (ChronoUnit.DAYS.between(start, today) + 1) + ")> " +
                    message + System.lineSeparator();
        logbook.write(entry);
    }
}
```

Besser ist es, wenn man beide trennt:

```
class Mission {
    Logbook logbook;
    LocalDate start;
    void update(String author, String message) {
         final LocalDate today = LocalDate.now();
         String entry = String.format("%S: [%tm-%<te-%<tY](Day %d)> %s%n",
                author, today,
                ChronoUnit.DAYS.between(start, today) + 1, message);
        logbook.write(entry);
    }
}
```

Damit wird es auch einfacher Anpassungen vorzunehmen.

Favor For each over For loops (L2) ➕

```
class LaunchChecklist {
    List<String> checks = Arrays.asList("Cabin Pressure", "Communication", "Engine");
    Status prepareForTakeoff(Commander commander) {
        for (int i = 0; i < checks.size(); i++) {
            boolean shouldAbortTakeoff = commander.isFailing(checks.get(i));
            if (shouldAbortTakeoff) {
                return Status.ABORT_TAKE_OFF;
            }
        }
        return Status.READY_FOR_TAKE_OFF;
    }
}
```

Der Index wird nur für den Zugriff auf das Array benutzt. Deswegen ist dies redundant und öffnet die Türen für weitere Fehlern.

```
class LaunchChecklist {
    List<String> checks = Arrays.asList("Cabin Pressure", "Communication", "Engine");
    Status prepareForTakeoff(Commander commander) {
        for (String check : checks) {
            boolean shouldAbortTakeoff = commander.isFailing(check);
```

```
            if (shouldAbortTakeoff) {
                return Status.ABORT_TAKE_OFF;
            }
        }
        return Status.READY_FOR_TAKE_OFF;
    }
}
```

Im obigen Code-Snippet ist es klarer, es wird nur über die Liste iteriert und der Index hat keine Funktion.

Favor functional over imperative style (L4) ➕

Java erlaubt sowohl eine imperative als auch funktionale Arbeitsweise. Es lohnt sich zu wissen, wann man das eine oder das andere benutzt. Bei Listen lohnt sich meistens das funktionale vorgehen. Wie in folgendem Beispiel:

```
class Inventory {
    List<Supply> supplies = new ArrayList<>();
    long countDifferentKinds() {
        List<String> names = new ArrayList<>();
        for (Supply supply: supplies) {
            if (supply.isUncontaminated()) {
                String name = supply.getName();
                if (!names.contains(name)) {
                    names.add(name);
                }
            }
        }
        return names.size();
    }
}
```

Die Funktion prüft, welche Elemente in der Liste vorkommen. Viel einfacher und übersichtlicher geht es über funktionale Programmierung:

```
class Inventory {
    List<Supply> supplies = new ArrayList<>();
    long countDifferentKinds() {
         return supplies.stream()
                    .filter(supply -> supply.isUncontaminated())
                    .map(supply -> supply.getName())
                    .distinct()
                    .count();
    }
}
```

Use collect for terminating complex streams (L4) ➕

Im folgenden Beispiel wird eine Map von einer Liste mit Elementen und deren Häufigkeit erstellt, welche eine besondere Eigenschaft haben.

```
class Inventory {
    List<Supply> supplies = new ArrayList<>();
    Map<String, Long> countDifferentKinds() {
        Map<String, Long> nameToCount = new HashMap<>();
         Consumer<String> addToNames = name -> {
            if (!nameToCount.containsKey(name)) {
                nameToCount.put(name, 0L);
```

```
            }
            nameToCount.put(name, nameToCount.get(name) + 1);
        };
        supplies.stream()
                .filter(Supply::isUncontaminated)
                .map(Supply::getName)
                .forEach(addToNames);
        return nameToCount;
    }
}
```

Leider ist dies nicht sehr schnell ersichtlich, viel einfacher wird es mit der Benutzung von Collect():

```
class Inventory {
    List<Supply> supplies = new ArrayList<>();
    Map<String, Long> countDifferentKinds() {
        return supplies.stream()
                .filter(Supply::isUncontaminated)
                .collect(Collectors.groupingBy(Supply::getName, Collectors.counting()));
    }
}
```

Literatur

- Westphal, Ralf; Lieser, Stefan: Clean Code Developer. Liskov Substitution Principle [Online]. URL: http://clean-code-developer.de/die-grade/gelber-grad/#Liskov_Substitution_Principle [Stand: 05.04.2018]
- Harrer, Simon & Lenhard, Jörg & Dietz, Linus. (2018). Java by Comparison: Become a Java Craftsman in 70 Examples.
- Bloch, Jushua: Effective Java. Pearson Education, 2017, S. 227-230.
- Munroe, Randall: XKCD, Exploits of a Mom [Online]. URL: https://www.xkcd.com/327/ [Stand: 05.03.2018]
- Open Web Application Security Project: SQL Injection Prevention Cheat Sheet [Online]. URL: https://www.owasp.org/index.php/SQL_Injection_Prevention_Cheat_Sheet [Stand: 05.03.2018]